TALLER PRÁCTICO GRUPAL

Introducción a Git: Instalación, Configuración y Repositorio Local

Docente: Jhonathan Arley Peñaloza Sierra

Integrantes del grupo:	Yeimer Torres, Sofia Salazar, Crisbely Graterol
Fecha:	23/04/2026

Objetivo del taller

Practicar la instalación, configuración y uso básico de Git como sistema de control de versiones. Al finalizar, cada grupo habrá creado su primer repositorio local, realizado commits y explorado el historial de cambios.

Instrucciones generales

Formen grupos de 3 a 4 estudiantes. Cada grupo debe trabajar en un mismo computador. Sigan los ejercicios en orden, ya que cada parte construye sobre la anterior. Todos los integrantes deben participar. Entreguen este documento completo al finalizar.

Requisitos previos

Tener un computador con acceso a internet para descargar Git. Se recomienda tener Visual Studio Code instalado. No se necesita experiencia previa con Git.

Parte 1: Entendiendo los Conceptos de Git

Ejercicio 1.1 – ¿Qué es Git?

Respondan con sus propias palabras las siguientes preguntas basándose en lo visto en clase:

a) ¿Qué es un Sistema de Control de Versiones Distribuido (SCVD)? ¿Para qué sirve?

Es un sistema que permite guardar y controlar un proyecto ya sea código o documento

b) Mencionen al menos 3 cosas que permite hacer un sistema de control de versiones:

1. Guardar los cambios de un proyecto
2. Crea ramas para desarrollar nuevas ideas sin afectar el proyecto principal
3. Pueden trabajar varias personas al mismo tiempo sin perder información

Ejercicio 1.2 – Relacionar conceptos

Unan cada concepto de Git con su definición correcta. Escriban la letra correspondiente en la columna derecha:

> **Definiciones disponibles**
>
> A) Copia de un repositorio remoto a tu máquina local | B) Instantánea del proyecto en un momento determinado | C) Versión del repositorio que diverge del desarrollo principal | D) Área intermedia donde se colocan cambios antes de confirmarlos | E) Enviar cambios locales a un repositorio remoto | F) Propuesta de cambios para que el propietario revise y fusione | G) Archivo que especifica qué archivos Git debe ignorar | H) Incorporar cambios de una rama a otra

Concepto	Letra de la definición
`Commit`	B
`Branch`	C
`Staging Area`	D
`Push`	E
`Clone`	A
`Merge`	H
`Pull Request`	F
`.gitignore`	G

Parte 2: Instalación y Configuración de Git

Ejercicio 2.1 – Instalar Git

Sigan los pasos de instalación según su sistema operativo (vistos en clase). Cuando terminen, verifiquen que Git está instalado.

a) Abran una terminal (CMD, PowerShell o Terminal de Linux) y escriban el siguiente comando:

```
git --version
```

b) ¿Qué versión de Git les aparece? Escríbanla aquí:

git version 2.53.0.windows.2

c) ¿Tuvieron algún problema durante la instalación? Si es así, descríbanlo y cómo lo resolvieron:

No, no tuvimos ningún inconveniente

Ejercicio 2.2 – Configuración inicial

Ahora van a configurar Git con su nombre de usuario y correo electrónico. Estos datos aparecerán en cada commit que hagan.

a) Ejecuten los siguientes comandos en la terminal, reemplazando los datos con los suyos:

```
# Configurar nombre de usuario
git config --global user.name "SuNombreDeUsuario"

# Configurar correo electrónico
git config --global user.email "sucorreo@ejemplo.com"

# (Opcional) Configurar VS Code como editor
git config --global core.editor "code --wait"
```

b) ¿Con qué nombre y correo configuraron Git? Escríbanlos:

Yeimer Torres – yeimertorres30@gmail.com
Sofia Salazar – sof71101@gmail.com
Crisbely Graterol – graterolcrisbely@gmail.com

> **Recuerden**
> Se aconseja tener una cuenta de usuario en GitHub para facilitar la colaboración y la conexión remota con repositorios en la plataforma.

Parte 3: Creando su Primer Repositorio Local

Ejercicio 3.1 – Crear un proyecto e inicializar Git

Van a crear una carpeta de proyecto y a inicializar Git en ella. Sigan estos pasos exactamente:

Paso 1: Crear la carpeta del proyecto

Creen una carpeta en el escritorio llamada "mi-primer-repo". Pueden hacerlo desde el explorador de archivos o con la terminal.

Paso 2: Abrir la carpeta en VS Code

Abran Visual Studio Code y abran la carpeta que acaban de crear (Archivo → Abrir carpeta).

Paso 3: Abrir la terminal integrada

En VS Code, abran la terminal integrada (menú Terminal → Nueva terminal, o Ctrl + Ñ).

Paso 4: Inicializar Git

Escriban el siguiente comando:

```
git init
```

a) ¿Qué mensaje les apareció en la terminal después de ejecutar git init?

Initialized empty Git repository in C:/Users/Yeimer/Downloads/mi-primer-repo/.git/

b) ¿Qué cambios observan en VS Code después de inicializar Git? (Miren la barra inferior y la barra lateral)

Aparece la rama en la que estamos en este caso se llama master

> **Dato importante**
> Después de git init, aparece la palabra "main" en la barra inferior. Es la rama principal que Git crea por defecto. Los archivos aparecen con la letra U (Untracked) porque Git aún no los está rastreando.

Ejercicio 3.2 – Crear archivos para el proyecto

Ahora van a crear 3 archivos dentro de su proyecto para tener contenido con el cual practicar:

a) Creen los siguientes archivos en VS Code (Archivo → Nuevo archivo):

Archivo	Contenido sugerido
README.md	`# Mi Primer Repo\nEste es mi primer proyecto con Git.`
index.html	`<html><body><h1>Hola Git</h1></body></html>`
notas.txt	`Aquí van mis notas de clase sobre Git.`

b) Después de crear los archivos, ejecuten en la terminal:

```
git status
```

c) ¿Qué muestra git status? ¿De qué color aparecen los archivos? ¿Qué significa eso?

On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        index.html
        notas.txt

nothing added to commit but untracked files present (use "git add" to track)

Parte 4: Gestionando el Staging Area

Ejercicio 4.1 – Agregar archivos al Staging Area

Ahora van a mover archivos al área de preparación (staging area) para prepararlos antes de hacer un commit.

a) Agreguen SOLO el archivo README.md al staging area:

```
git add README.md
```

b) Ejecuten git status de nuevo. ¿Qué diferencia ven ahora entre README.md y los demás archivos?

On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file: README.md

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html
        notas.txt

---

c) Ahora agreguen TODOS los archivos restantes de una sola vez:

```
git add .
```

d) Ejecuten git status nuevamente. ¿Qué letra aparece ahora al lado de cada archivo? ¿Qué significa?

On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file: README.md
        new file: index.html
        new file: notas.txt

Ejercicio 4.2 – Eliminar un archivo del Staging Area

A veces agregamos un archivo por error. Practiquen cómo sacarlo del staging area:

a) Saquen el archivo notas.txt del staging area:

```
git rm --cached notas.txt
```

b) Ejecuten git status. ¿En qué estado quedó notas.txt? ¿Y los demás archivos?

On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file: README.md
        new file: index.html

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        notas.txt

---

c) Vuelvan a agregar notas.txt para que todos los archivos estén listos para el commit:

```
git add notas.txt
```

> **Resumen de estados**
> U (Untracked) = Git no rastrea ese archivo aún | A (Added) = El archivo está en el staging area, listo para commit | M (Modified) = El archivo fue modificado después del último commit

Parte 5: Realizando Commits

Ejercicio 5.1 – Su primer commit

Con todos los archivos en el staging area, van a guardar su primer instantánea del proyecto:

a) Ejecuten el siguiente comando para hacer su primer commit:

```
git commit -m 'feat: crear estructura inicial del proyecto'
```

b) ¿Qué mensaje les mostró la terminal después del commit? Escríbanlo:

[master (root-commit) be55c33] feat: crear estructura inicial del proyecto
 3 files changed, 3 insertions(+)
 create mode 100644 README.md
 create mode 100644 index.html
 create mode 100644 notas.txt

Ejercicio 5.2 – Modificar, agregar y hacer un segundo commit

Ahora van a experimentar el flujo completo: modificar → agregar al staging → hacer commit.

a) Modifiquen el archivo index.html: agreguen un párrafo debajo del h1. Guarden el archivo.

b) Ejecuten git status. ¿Qué letra aparece al lado de index.html ahora? ¿Por qué?

On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified: index.html

no changes added to commit (use "git add" and/or "git commit -a")

Aparece asi pq no hemos modificado el archivo en el repositorio

---

c) Agreguen el archivo y hagan un commit con un mensaje usando conventional commits:

```
git add index.html
git commit -m 'su mensaje aquí usando conventional commits'
```

d) ¿Qué tipo de conventional commit usaron y por qué? (feat, fix, refactor, docs, style, chore)

Usamos feat ya que indica que agregamos una caracteristica al archivo index.html

Ejercicio 5.3 – Conventional Commits

Para cada situación, escriban qué tipo de conventional commit usarían:

Situación	Tipo de commit
Corregí un error en una función de Python	fix
Agregué una nueva página al sitio web	feat
Mejoré los nombres de las variables del código	refactor
Actualicé el archivo README.md con instrucciones	docs
Cambié los colores del CSS de la página	style
Reorganicé las carpetas del proyecto	chore

Parte 6: Explorando el Historial de Commits

Ejercicio 6.1 – Ver el historial

a) Ejecuten el siguiente comando para ver todos sus commits:

```
git log
```

b) Escriban la información que muestra para cada commit (ID, autor, fecha, mensaje):

commit 2b023e59c1036050f54e15413b63f43fef1d3a7f (HEAD -> master)
Author: Yeimer Torres <yeimertorres30@gmail.com>
Date: Thu Apr 23 20:26:13 2026 -0500

    su mensaje aquí usando conventional commits

commit be55c332416b06376c60d2d8a036c420d21cb7b9
Author: Yeimer Torres <yeimertorres30@gmail.com>
Date: Thu Apr 23 20:23:46 2026 -0500

    feat: crear estructura inicial del proyecto

---

c) Ahora prueben la versión resumida y compárenla:

```
git log --oneline
```

d) ¿Qué diferencia hay entre git log y git log --oneline? ¿Cuál les parece más útil y por qué?

La diferencia es que git log –oneline da la misma información pero de manera mas resumida y es mejor ya que es mas útil para navegar rapido

---

Ejercicio 6.2 – Navegar entre commits

Van a viajar en el tiempo para ver cómo estaba su proyecto en el primer commit:

a) Copien el ID (los primeros caracteres) de su PRIMER commit del git log --oneline y ejecuten:

```
git checkout <peguen-aqui-el-ID-del-primer-commit>
```

b) ¿Qué pasó con los archivos del proyecto? ¿Se ven los cambios que hicieron después?

Los archivos muestran solo lo que existía en el commit be55c33, y no verás los cambios que se hicieron después en master

---

c) Vuelvan a la rama principal ejecutando:

```
git checkout main
```

d) ¿Se recuperaron todos los archivos y cambios? ¿Qué aprendieron de esta experiencia?

Sí, se recuperaron todos los archivos y cambios que se habían hecho después del primer commit.
Cada commit guarda un estado completo del proyecto: puedes moverte hacia atrás o adelante en el tiempo.

> **Importante**
> HEAD siempre apunta al último commit de la rama actual. Los cambios en commits anteriores se perderán si no se asocian a una nueva rama con: git checkout -b nombreRama

Parte 7: Reflexión Grupal

Ejercicio 7.1 – ¿Por qué usar Git?

Ahora que ya practicaron, respondan como grupo:

a) De los beneficios vistos en clase, ¿cuál les parece el más importante y por qué?

Que pueden trabajar varias personas al mismo tiempo sin perder información

b) Imaginen que están trabajando en un proyecto grupal de programación SIN Git. ¿Qué problemas podrían tener?

Que se tardaría mas en terminar los proyectos e incluso se podrían perder parte de la información

c) Escriban en orden los comandos que aprendieron hoy, creando una "cheat sheet" de Git básico:

Comando	¿Qué hace?
`git init`	Para inicializar el repositorio
`git status`	Para ver que cambio en los comits
`git add .`	Para preparar los cambios del staging area
`git rm --cached`	Quita un archivo del staging area en el historial del repositorio como un mensaje
`git commit -m`	Guarda los cambios del staging area en el historial del repositorio con un mensaje
`git log`	Muestra el historial completo de los commits ¿: autor, fecha, hash y mensaje de cada uno
`git log --oneline`	Muestra el historial de commits resumido
`git checkout`	Cambia de rama o restaura archivos

Criterios de Evaluación

- Instalación exitosa de Git y verificación con git --version
- Configuración correcta de user.name y user.email
- Creación del repositorio local con git init y archivos del proyecto
- Manejo correcto del staging area (git add, git rm --cached, git status)
- Realización de commits con mensajes usando conventional commits
- Exploración del historial con git log y navegación con git checkout
- Calidad de las reflexiones grupales y la cheat sheet final
- Participación activa de todos los integrantes del grupo